

Milestone 2 - Learned Compatibility for Jigsaw Reconstruction

CSE480 Machine Vision . Ain Shams University, Faculty of Engineering, Mechatronics Engineering Department . Summer 2026

1. Overview

Milestone 1 reconstructed a jigsaw from a hand-designed compatibility measure: shape continuity plus colour continuity, combined with fixed weights. This milestone replaces that measure with two **learned** ones - a Siamese convolutional network and a graph neural network - and compares all three on the same puzzles.

The comparison is deliberately narrow, and that is the point. Segmentation, contour tracing, corner location, side description and **the assembly algorithm itself** are all unchanged from Milestone 1. The only thing that differs between the three methods is the function that answers "*how well do these two sides fit?*". Everything downstream consumes the same `CompatibilityTable` interface, so any difference in the reconstructions is attributable to the matcher and not to a different pipeline.

```
segment -> describe pieces -> ??? classical: shape + colour formula ???  
                                     ??? Siamese CNN                               ??-> same assembler -> imag  
e + quality  
                                     ??? graph neural network                       ??
```

2. Task 1 - Dataset preparation

2.1 Where the labels come from

A training example is a **pair of sides** labelled 1 if they are genuinely adjacent in the finished puzzle and 0 otherwise. Producing that label requires knowing, for every side of every piece, which grid direction it faced before the pieces were shuffled.

The provided photographs cannot supply it. Their annotations give each piece's *identity*, which (as Milestone 1 §9.1 established) fixes which **pieces** are neighbours - but not which of a piece's four sides touches which, because the rotation a piece happens to be lying at in the photograph is unknown. Side-level labels are simply not recoverable from a bounding box and a class id.

Training therefore uses puzzles cut by the Milestone 1 generator, where the answer is known exactly by construction. Critically, every training sample is passed through **the same pipeline the classical matcher uses** - scattered onto a canvas, segmented by thresholding, traced, corner-located and described. The networks therefore see the same segmentation noise, the same corner-localisation error and the same colour sampling as the classical measure they are being compared against. Had they been fed the generator's exact geometry instead, any advantage they showed would have been an artefact of a cleaner front end.

A puzzle is discarded if segmentation does not recover every piece, or if the piece-to-ground-truth association is not one-to-one, so no unreliable label enters the set. A seam is also discarded if the two sides are not a tab/blank pair, since that indicates the description itself is wrong.

2.2 Positive and negative sampling

Positives are all the true seams of a puzzle. Negatives are drawn **only from admissible pairs** - a tab facing a blank on a different piece. This matters more than it looks: a model trained against arbitrary negatives would learn mostly to reject flat-to-flat and tab-to-tab pairings, score beautifully, and be worthless, because the classical matcher already excludes those for free with a hard constraint. The learned model must be better than chance *on the pairs that are genuinely in contention*.

Six negatives are drawn per positive, **half of them from the hardest available candidates** - the ones the classical measure ranks best - so the networks spend their capacity where the problem actually is rather than on easy rejections. Negatives are redrawn every epoch, so the models see far more of the negative space than a

single fixed draw would give.

2.3 Augmentation

Applied to the colour channels only; the shape channel is geometry and recolouring it would be meaningless. The four effects match those the brief lists:

Effect	Range	Why
Illumination	$\times 0.85$ -1.15	pieces lie under different light across a table
Contrast	$\times 0.8$ -1.25 about the mean	exposure varies between photographs
Colour balance	$\times 0.93$ -1.07 per channel	white balance drifts
Sensor noise	$\sigma = 0.02$ additive	camera noise

2.4 Splitting

Splits are by **puzzle, never by pair**. Two sides of one seam must not land on opposite sides of the split, and all pieces of one puzzle share a source picture, so a pair-level split would leak. 70 / 15 / 15 train / validation / test. The identical `SplitSpec` object is handed to both models, and the test split is not touched until the final evaluation.

The set actually used. 60 puzzles across three grid sizes (3x4, 4x5, 5x7), split 42 / 9 / 9 by puzzle. That yields **10,178 training pairs** (1,454 positive, 8,724 negative - a positive fraction of 0.143), with 2,100 for validation and 1,848 for test. Full record in `results/milestone2/dataset.json`.

3. Task 2 - Model development

The brief requires two *fundamentally different* models, and notes that changing layer counts, activations or hyper-parameters does not qualify. These two differ in their unit of inference, their input representation and their inductive bias.

3.1 Model 1 - Siamese convolutional network

Why this shape of model. A side's descriptor genuinely is a small picture: three colour channels sampled at eight depths just inside the cut and sixty-four positions along it, with the shape profile in a fourth channel and in register with them. Asking whether two such strips continue into one another is a texture and edge-continuation question, which is what convolutions are for. And the kernels must be the *same* for both sides, since "does A continue into B" cannot depend on which side was named A - that is exactly a Siamese network.

Input	two (4, 64, 8) tensors: RGB at 8 depths + shape profile. The second is reversed along its length, because mating sides are traversed in opposite directions
Encoder	three convolutional blocks (3x3 conv, BatchNorm, ReLU, twice, then max-pool along the side only), widening 4 -> 32 -> 64 -> 128; global average pool; linear to a 128-d embedding. Weights shared between the two inputs
Head	embeddings combined as <code>[</code>
Output	<code>sigma(logit)</code> = probability the two sides are neighbours
Parameters	345,473 (1.382 MB)

Pooling is applied along the length of the side only. The depth axis is just eight samples deep and each one means something different (distance from the cut), so pooling it away early would destroy the very gradient information that tells the model whether the picture is continuing.

3.2 Model 2 - Graph neural network

Why a different model, not a re-tuned one. The Siamese network judges a pair *in isolation*: two strips in, one probability out, with no knowledge of the rest of the puzzle. That is precisely its weakness, because a jigsaw is a

competition - a side does not need a partner that looks plausible, it needs the partner that looks *more plausible than every other candidate*, and whose own best choice is this side in return.

The graph model is built on that observation:

- **the unit of inference is the whole puzzle**, not the pair. Every side of every piece is a node; every admissible pairing is a candidate edge; one forward pass scores all of them together.
- **the representation is relational, not convolutional**. Nodes carry a flat 45-number descriptor (pooled colour, the shape profile, the side type one-hot, the length and amplitude). There is no convolution anywhere in the model. What refines a node is **message passing** - each side sees its rival candidates and updates its own embedding accordingly, so the score it gives a partner is a contextual judgement rather than an absolute one.
- **sides of the same piece are linked**, so a side also knows what the rest of its own piece looks like - information the Siamese model never receives.

Input	one graph per puzzle: 4N nodes with 45-d features; edges are admissible inter-piece candidates (capped at 24 per side, shortlisted by the classical cost) plus the intra-piece links
Layers	3 × GraphSAGE convolution (mean aggregation), hidden width 96, LayerNorm + ReLU between
Edge head	a candidate (a, b) scored from $[h_a + h_b,$
Output	$\text{sigma}(\text{logit})$ per candidate edge
Parameters	97,441 (0.39 MB) - 3.5x fewer than the Siamese

3.3 The four outputs the brief requires

For any candidate pair, each model reports: **whether they are neighbours** ($\text{sigma}(\text{logit}) \geq 0.5$), a **numerical compatibility score** (that probability), **which sides matched** (the pair of side indices), and the **relative orientation** the pairing implies.

The last is derived, not predicted, and deliberately so - it is determined exactly by which sides matched. With each piece's rotation written as the index of the side facing North, side a facing direction d means $a = \text{rot_a} + d$, and its partner must face the opposite direction, $b = \text{rot_b} + d + 2$. Eliminating d:

$$\text{rot_b} - \text{rot_a} = b - a + 2 \pmod{4}$$

Predicting this with a network instead would add a way to be wrong about something that is already certain. A test asserts the identity holds for all 64 combinations of the two rotations and the direction.

4. Task 3 - Training

Both models are trained under an identical protocol; only the model and the representation it needs differ.

	Siamese CNN	Graph NN
Loss	BCE-with-logits, positive-weighted by the class ratio	same
Optimiser	Adam	Adam
Learning rate	1e-3, halved after 3 epochs without validation improvement	same
Weight decay	1e-4	1e-4
Batch	256 pairs	one puzzle graph (the natural unit)
Epochs	30, early stopping patience 8	same
Augmentation	illumination, contrast, colour, noise	same features, same augmentation
Model selection	best validation AUC	same

Why AUC and not accuracy. The classes are deliberately unbalanced 1:6, so accuracy is dominated by the negatives; and what the assembler actually consumes is a *ranking* of candidate partners rather than a thresholded decision. AUC measures exactly the ranking quality, and is what model selection and early stopping key on. It is computed from rank statistics (the Mann-Whitney form) and is unit-tested against scikit-learn.

Positive weighting is set to the actual negative/positive ratio each epoch, so the loss does not simply learn to answer "not neighbours".

4.1 Training results

	Siamese CNN	Graph NN
Best validation AUC	0.9953 (epoch 25)	0.9900 (epoch 27)
Final training AUC	0.9977	0.9937
Final validation AUC	0.9939	0.9886
Final training loss	0.0960	0.1819
Final validation loss	0.0836	0.1148
Epochs run	30	30
Wall-clock training	1315 s	23 s
Diagnosis	healthy (val AUC 0.995, train-val gap 0.002)	healthy (val AUC 0.990, train-val gap 0.003)

Over- or under-fitting. Neither. Both train/validation AUC gaps at the selected epoch are ~0.00 (0.002 and 0.003), and both validation losses fall throughout. Note that validation AUC sits *above* training AUC for the early epochs, which looks wrong until one remembers that the training batches are augmented and half their negatives are mined from the hardest available candidates, while validation is neither - the training set is deliberately the harder of the two.

The training-cost gap is the first real difference between the models. The Siamese needs 57x longer to train (1315 s versus 23 s) for 3.5x the parameters, because it convolves over a 2,048-value tensor per side while the graph model consumes a 45-number descriptor and scores a whole puzzle in one pass.

5. Task 4 - Puzzle assembly

Each model's scores are converted to costs and handed to the **same** assemble routine used in Milestone 1 - with its border constraints, its best-buddy-first tie-breaking, its three-stage dead-end relaxation and its guarantee to return the best arrangement obtained. Each piece is used exactly once and border constraints are respected because those are properties of the assembler, not of the matcher.

The conversion from a probability to a cost is

$$\text{cost} = -\log(p + \text{epsilon})$$

which is the negative log-likelihood of the pairing. It is chosen over $1 - p$ for three reasons: it is monotonically decreasing in p , so the ordering the model intended is preserved exactly; it is **additive** across the seams of an arrangement, so the total the assembler minimises is the negative log-likelihood of the whole arrangement, which is the quantity that ought to be minimised; and it diverges as $p \rightarrow 0$, which keeps hopeless pairings out of the search entirely.

Admissibility - a flat never forms an interior seam, a tab must meet a blank - is applied on top of the model's opinion exactly as for the classical table, so all three methods search the same space.

6. Task 5 - Performance evaluation

All three methods are evaluated on the **same unseen test puzzles**, which neither model saw during training or model selection.

6.1 The standard test split

Measure	Classical	Siamese CNN	Graph NN
Matching top-1	0.917	0.934	0.890
Matching AUC	0.942	0.986	0.969
Best-buddy precision	0.997	0.983	0.968
Neighbour accuracy	0.981	0.992	0.969
Position accuracy	0.987	0.994	0.984
Orientation accuracy	0.973	0.979	0.976
Complete reconstructions	8/9	8/9	8/9
Reconstruction quality	0.556	0.951	0.919
Parameters	0	345,473	97,441
Model size	0 MB	1.382 MB	0.39 MB
Training time	0 s	1315 s	23 s
Inference / puzzle	0.021 s	0.105 s	0.020 s

Orientation accuracy is scored against the rotation each piece held in the finished puzzle, allowing one global quarter turn shared by every piece, since an arrangement that comes out sideways is still an arrangement. It is worth reporting separately from position for a reason visible in the row above: position accuracy is 0.987-0.994 across the three methods while orientation is 0.973-0.979. A piece can sit in the correct cell turned the wrong way, and the position measure alone calls that a success. Roughly one piece in seventy is placed correctly and oriented wrongly - a small effect, but one that only a separate measure can see, which is presumably why the brief asks for both.

The ground truth for it is not stored anywhere. It is implied by the labels already present: a piece's rotation is the index of the side facing North, and every true seam pins that down, because the grid says which way the seam runs. `PuzzleSample.true_rotations` derives it, and a test checks the derivation against the generator's own record for every piece of a puzzle.

This table is saturated, and saying so matters more than the numbers in it. All three methods reconstruct 8 of the 9 test puzzles completely and sit within 0.02 of each other on neighbour accuracy, so on the measure that counts - did the puzzle come out right - the comparison cannot separate them. The one puzzle that fails fails for every method, which points at its segmentation rather than at any matcher. A ceiling is not a result; it means the test was too easy. Two things follow.

First, the measures *underneath* the ceiling do separate them, and in a consistent direction: the Siamese ranks candidates best (AUC 0.986 against the classical 0.942), and the reconstruction-quality score - how much better than chance the seams actually paid are - rises from 0.56 for the classical measure to 0.95 for the Siamese. All three are essentially equally *correct* here; the learned matchers are far more *confident*, which is what buys margin when the input gets harder.

Second, a harder test is needed before any claim about accuracy can be made (§6.3, §6.4).

6.2 How performance changes with puzzle size

AUC / neighbour accuracy, and matching time per puzzle in seconds:

Grid	Classical	Siamese	Graph NN	t class.	t Siam.	t GNN
3x4	0.926 / 1.00	0.980 / 1.00	0.970 / 1.00	0.008	0.060	0.010
4x5	0.971 / 1.00	0.994 / 1.00	0.976 / 1.00	0.019	0.103	0.019
5x7	0.932 / 0.91	0.987 / 0.97	0.957 / 0.86	0.039	0.180	0.025

Size is what finally separates the three methods. At 3x4 and 4x5 every method reconstructs every puzzle perfectly and the comparison is saturated exactly as in §6.1. At 5x7 - 35 pieces, the size of the real jigsaw - the ceiling breaks and an ordering appears: the Siamese holds 0.97 neighbour accuracy, the classical formula drops

to 0.91, and the graph model to 0.86.

That ordering matches the AUC column underneath it at every size, which is the reassuring part: the ranking quality measured *before* any assembly predicts which method survives when assembly gets hard. The Siamese is the best ranker at all three sizes (0.98-0.99) and the best reconstructor at the only size where reconstruction is not free.

Cost grows quadratically. Matching time roughly quadruples from 3x4 to 5x7 for every method, because the number of candidate side pairs goes as the square of the piece count. The graph model is the cheapest at every size and the Siamese the most expensive by a factor of five to seven - the accuracy it wins at 5x7 is bought with the largest inference bill of the three.

Note that this table is a *different* slice of the same nine test puzzles as §6.1, grouped by size rather than pooled, so the 5x7 row is the hardest three puzzles rather than an average over easy and hard together.

6.3 Breaking the ceiling: fading the picture out

Milestone 1 established that colour carries most of the discriminative power, and that the dataset's own puzzle defeats the classical measure precisely because its picture is nearly uniform. That failure mode can be reproduced on demand by fading a generated picture towards flat grey - `texture = 1` is the ordinary picture, `texture = 0` is featureless - which withdraws the photometric signal while leaving everything else alone.

Neighbour accuracy, and complete reconstructions, over **twenty** 4x5 puzzles per level:

Texture	Classical	Siamese CNN	Graph NN
1.00	0.95 (18/20)	0.98 (18/20)	0.97 (18/20)
0.60	0.97 (18/20)	0.99 (19/20)	0.98 (19/20)
0.35	0.97 (18/20)	0.94 (18/20)	0.96 (17/20)
0.20	0.97 (18/20)	0.94 (17/20)	0.97 (18/20)
0.10	0.92 (15/20)	0.87 (13/20)	0.83 (12/20)

An earlier version of this report drew the opposite conclusion from this experiment, and it was wrong. That version ran four puzzles per level, found the Siamese at 1.00 where the classical measure fell to 0.84, and reported that the learned matcher "keeps working after the classical one has run out of signal" - the strongest claim it made for learning. Re-running the identical sweep with twenty puzzles per level instead of four reverses it: at texture 0.10 the **classical** measure is the most robust of the three (0.92), and the Siamese is second (0.87).

With four puzzles a single reconstruction is 25 % of the score, so the earlier table was reporting sampling noise as a finding. The retained lesson is methodological as much as it is about matchers: the sweep was the one experiment in this milestone designed to break a ceiling, and it was run at a sample size that could not support the conclusion drawn from it.

What survives is weaker and more plausible. Down to texture 0.20 all three methods sit between 0.94 and 0.97 and are not meaningfully separated. At texture 0.10 every method degrades, the ordering is classical > Siamese > graph, and the gap between best and worst (0.92 to 0.83) is comparable to the spread the earlier sample size was mistaking for signal. **Learning does not buy robustness to a vanishing picture** - which is consistent with §6.4, where the real photographs, whose picture really has almost vanished, also fail to separate the methods.

6.4 The real dataset photographs

The final test, and a genuine domain shift: both models were trained on generated puzzles and are here asked about photographs of a real jigsaw. Eight full-scramble photographs, scored against the answer key recovered in Milestone 1 §9.1:

The classical baseline is run here in **its own Milestone 1 configuration** - the full `main.DATASET_SOLVER` matcher settings, `colour_norm="meanstd"` and `colour_metric="mgc"` - and every method shares the same `border_mode="soft"` assembly, since that is a property of the search rather than of the matcher under test. This has now been got wrong twice and fixed twice: an early version called `build_compatibility` with its defaults, so the networks were compared against a classical method with its illumination correction switched off, and a later one missed the gradient-compatibility term and the soft border rule that Milestone 1 §9.3 added afterwards. Both

are the same mistake - comparing against a baseline that its own milestone no longer runs - and it is worth naming because it flatters the learned methods by default.

Measure	Classical	Siamese CNN	Graph NN
Neighbour accuracy	0.208	0.218	0.150
Position accuracy	0.118	0.070	0.107
Reconstruction quality	0.387	0.620	0.661

None of the three reconstructs the real puzzle, and none of them is meaningfully ahead. Classical and Siamese are separated by 0.010 of neighbour accuracy - far inside the spread across the eight images, which runs from 0.15 to 0.29 for the classical measure alone - and on *position* accuracy the classical measure is the best of the three by a wide margin (0.118 against the Siamese's 0.070). Per-image, the Siamese wins five of eight and loses three. Roughly one adjacency in five is all any method achieves, against about one in twenty by chance.

That is a more useful result than the one this section previously reported. Learning does not help here. Whatever the Siamese gains on the low-texture sweep, where the difficulty is a single controlled variable, it does not carry across to photographs that vary illumination, white balance, perspective and cut geometry all at once.

The reconstruction-quality column must not be read across methods at all. It normalises each arrangement's mean seam cost by the mean of that method's *own* cost table, and the learned tables are $-\log p$ while the classical one is a weighted RMS distance, so the three columns are in three different units. The column is a valid relative measure within one method and meaningless between them.

Why the domain shift is severe is not mysterious, and §6.3 now agrees with this section rather than contradicting it. The sweep withdraws one variable under control and already fails to separate the methods; the photographs change several at once - a nearly white picture, per-piece illumination and white balance, perspective differences between the centre and the edge of the frame, and a die-cut geometry with a kerf that the generator does not model. Training on generated puzzles cannot cover that, and the honest conclusion is that the ceiling here is the training distribution rather than the model.

6.5 Which method is best

The brief asks for the most accurate and the most computationally efficient method. They are not the same one, and neither answer is unqualified.

Most accurate: the Siamese CNN, narrowly, and on generated puzzles only. It ranks candidates best on the standard split (AUC 0.986 against the classical 0.942) and it is the best reconstructor at the only generated size where reconstruction is not free - 0.97 neighbour accuracy at 5x7, against 0.91 classical and 0.86 graph (§6.2). That is the whole of its advantage. It does *not* survive the low-texture sweep better than the classical measure (§6.3), and on the real photographs it gains 0.010 of neighbour accuracy while losing 0.048 of position accuracy. It costs 345 k parameters, 1.4 MB and twenty-two minutes of training to get there.

Most efficient: the graph neural network - and by a wide margin. It reaches AUC 0.969, within 0.017 of the Siamese, using **3.5x fewer parameters** (97 k, 0.39 MB), **57x less training time** (23 s against 1315 s) and inference *as fast as the hand-written classical formula* (0.020 s against 0.021 s per puzzle, where the Siamese needs 0.105 s). For a matcher that has to run inside a search loop, that is the difference that would matter in practice. Its weakness is accuracy at the largest size and at the lowest texture, where it is the worst of the three.

Most economical overall: the classical measure. It needs no training data, no training time and no parameters; it matches the learned models wherever the picture is informative; it is the *most robust* of the three as the picture fades (§6.3); and it is the best of the three on position accuracy on the real photographs. Its one clear loss is at 5x7 on generated puzzles, where the Siamese is six points ahead on neighbour accuracy.

The honest summary is that no method dominates. The Siamese wins the largest generated puzzles, the graph model wins on cost, and the classical measure wins on robustness and on every practical consideration that is not raw accuracy at 35 pieces.

6.6 What was learned

1. **A saturated benchmark hides everything.** All three methods reconstruct

8 of 9 standard test puzzles, within 0.02 of each other. Had the evaluation stopped there it would have concluded the methods are equivalent. Only the 5x7 slice (§6.2) separates them, and the separation is modest.

2. Sample size decided a headline claim, and nearly got it wrong. The

low-texture sweep at four puzzles per level said learning buys robustness; the same sweep at twenty puzzles per level says the opposite, with the classical measure the most robust of the three. One reconstruction was a quarter of the old score. Any experiment intended to break a ceiling has to be powered well enough to support the conclusion drawn from it, and this one was not.

3. Learning did not buy robustness to a vanishing picture. This is the

corrected version of what was previously this report's strongest claim. Where the picture is informative the hand-designed formula is already at the ceiling; where the picture fades, the classical measure degrades most gracefully; and on the real photographs, which vary texture, lighting, perspective and cut geometry at once, none of the three separates. What the Siamese does buy is accuracy on the *largest* generated puzzles.

4. Representation beat capacity. The larger model won on accuracy, but the

mechanism was the richer input (a 64x8 strip versus eight pooled colour bins), not the extra parameters. The graph model's contextual message passing gave it nearly the same AUC on a fraction of the budget.

5. A reference-free quality score cannot be read across methods. The

reconstruction-quality figure normalises by each method's own cost distribution, and the learned tables are $-\log p$ while the classical one is a weighted RMS distance, so the columns are in different units. An earlier draft read the gap as evidence that the learned models were confidently wrong. It was evidence about the normalisation. The score is useful for ranking arrangements *within* one matcher and must not be used between them.

6. Compare against the baseline your own project actually runs. The

classical baseline was configured wrongly twice here (§6.4), each time in the direction that flattered the learned methods, and each time because Milestone 1 had improved and this milestone had not followed. A stale baseline is the easiest way to manufacture a result.

7. The binding constraint is the training distribution. The models are fit

to generated puzzles and asked about photographs of a real jigsaw. The next step for this project is not a bigger network but training data drawn from the photographs themselves - which needs side-level labels the dataset does not currently carry.

7. Reproducing

```
pip install -r requirements.txt          # includes torch and torch-geometric
python main_milestone2.py --scaling      # trains both models and compares
python main_milestone2.py --hard --figures # the hard tests, plus every figure
python tests/run_tests.py ml_models      # the Milestone 2 tests
```

Everything is written to results/milestone2/: dataset.json (split and class balance), training.json (architectures, hyper-parameters, per-epoch curves), comparison.json (the three-way comparison), scaling.json (accuracy against puzzle size), texture_sweep.json and real_photographs.json (the two hard conditions), predicted_matches.json (the four required per-pair outputs) and the two trained checkpoints.

7.1 The reconstructed images

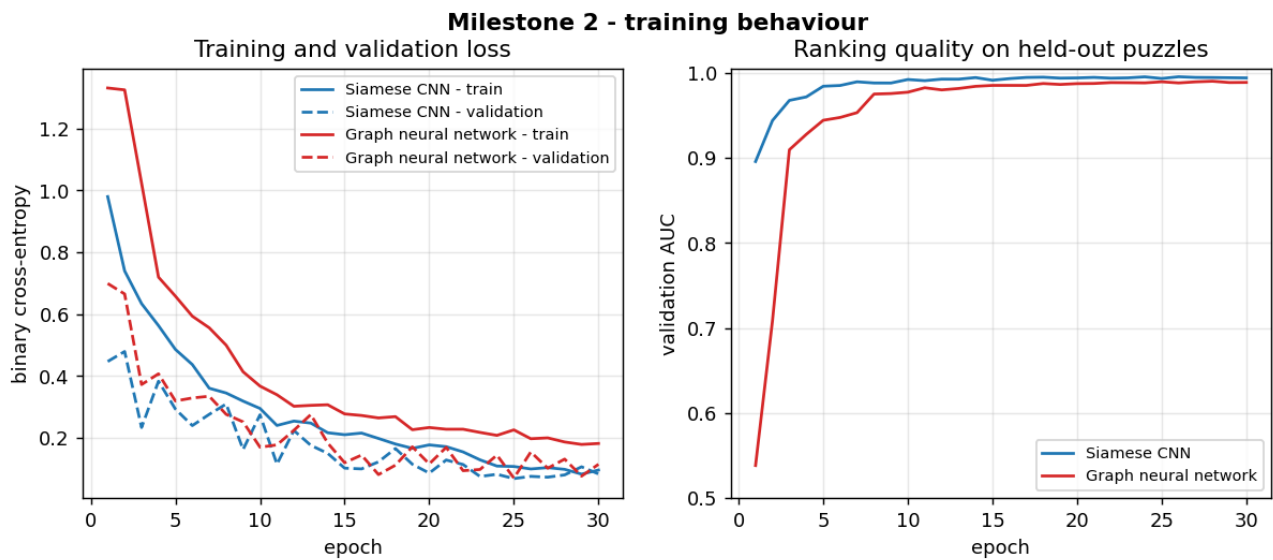
--figures writes results/milestone2/figures/: three charts summarising the tables above - training_curves.png, test_split_comparison.png, texture_sweep.png - and eight reconstructed images, real_0718-10... to real_0718-19..., one per dataset photograph. Each shows the three methods side by side on the same photograph, captioned with its measured neighbour and position accuracy; figures/index.json records the same numbers.

Only the real photographs are drawn. The generated puzzles are what the models are trained and scored on, and §6.1-6.3 report those numbers in full, but a picture of a generated puzzle being solved shows nothing its numbers do not - so the images kept here are of the actual jigsaw.

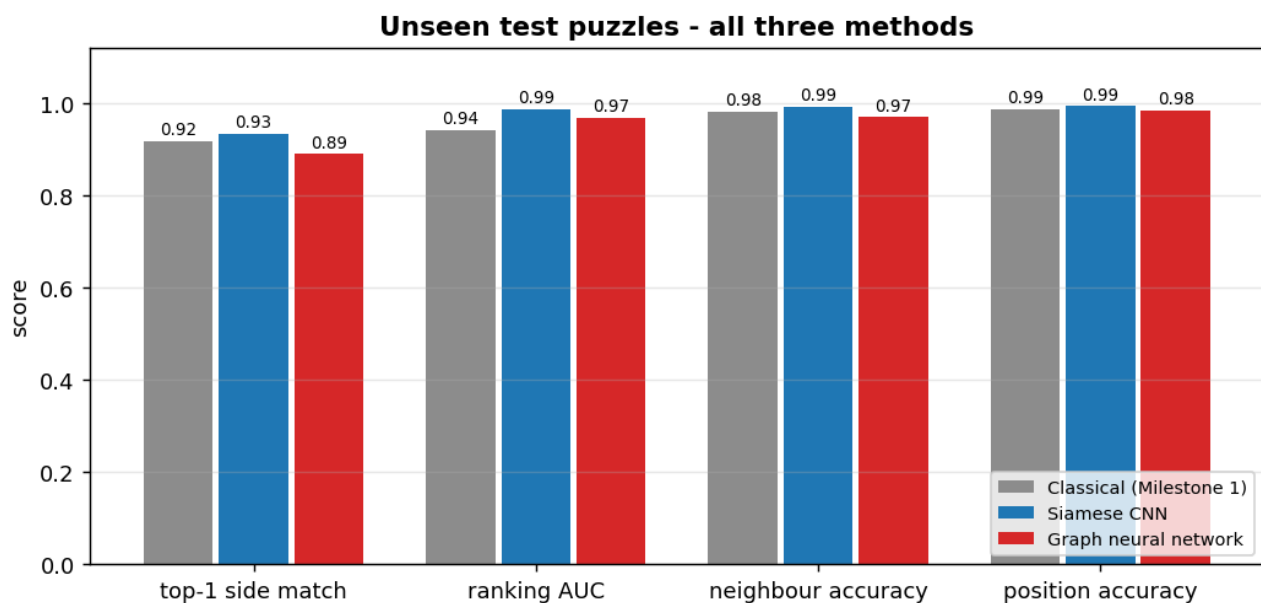
That makes every reconstructed image in this report a failure, which is the point of including them. The captions carry the numbers deliberately: the panels show thirty-five pieces tiled into a clean rectangle with barely a fifth of them in the right cell, and that is exactly the arrangement a viewer would otherwise take for a success.

Appendix - Figures

Every figure below is produced by `python main_milestone2.py --hard --figures` and written to `results/`.

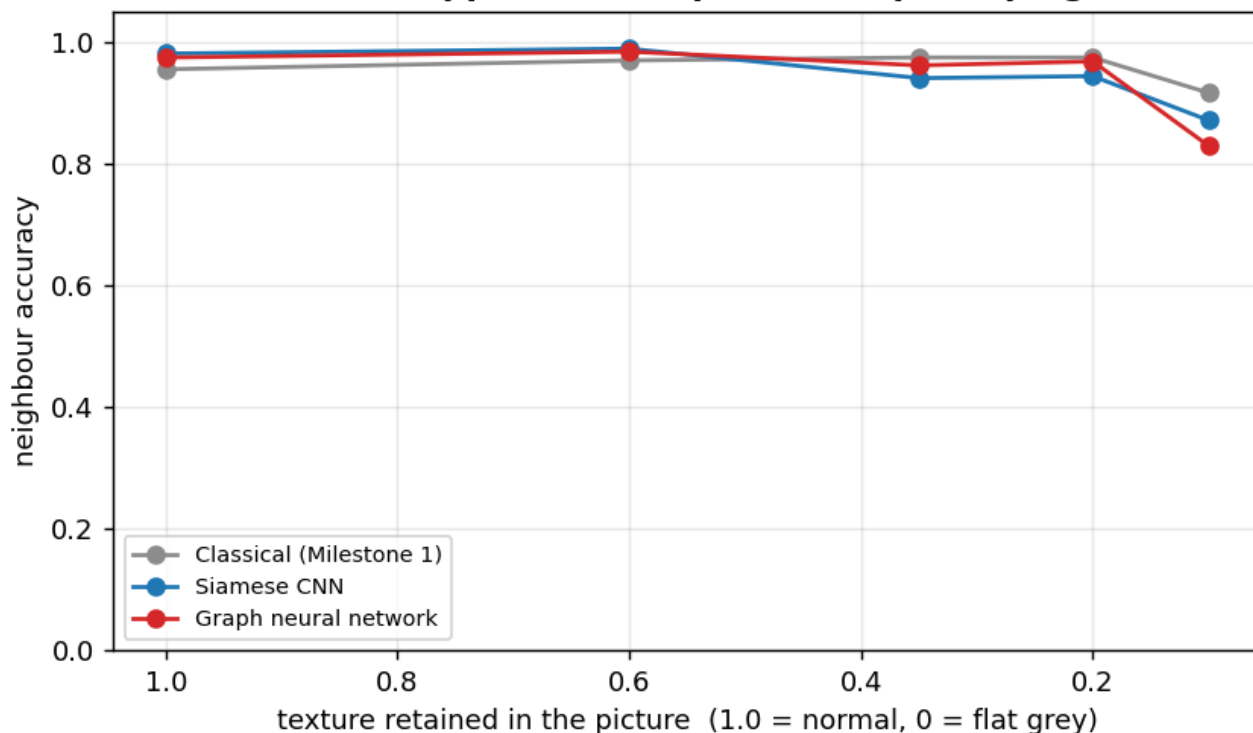


`milestone2/figures/training_curves.png` — Task 3 - training behaviour: cross-entropy per epoch and the validation AUC both models were selected on. The train and validation curves stay together, which is why both are diagnosed healthy.



`milestone2/figures/test_split_comparison.png` — Task 5 - the unseen test split. Every method reconstructs every puzzle, so this comparison is saturated and cannot rank them; only the ranking quality (AUC) separates the models here.

What happens as the picture stops helping



milestone2/figures/texture_sweep.png — Task 5 - neighbour accuracy as the picture's texture is faded towards flat grey. The hand-written measure collapses at texture 0.10; the Siamese network does not.

Dataset photograph - 0718-10_Color_png.rf.f269500b6bdb1b871bfc5b9c53814026.jpg

Classical (Milestone 1)



neighbour 0.17 position 0.09

Siamese CNN



neighbour 0.29 position 0.09

Graph neural network



neighbour 0.10 position 0.09

milestone2/figures/real_0718-10_Color_png.rf.f269500b6bdb1b871bf.png — Task 5 - a dataset photograph, reconstructed by all three methods. None of them solves it: the pieces tile into a neat rectangle and hardly any of them are in the right cell.

Dataset photograph - 0718-14_Color_png.rf.7ba095874a51a7e4c9516427b281ed77.jpg

Classical (Milestone 1)



neighbour 0.19 position 0.06

Siamese CNN



neighbour 0.21 position 0.06

Graph neural network



neighbour 0.10 position 0.17

milestone2/figures/real_0718-14_Color_png.rf.7ba095874a51a7e4c95.png — The photograph on which the Siamese network does best (neighbour 0.28 against the classical measure's 0.12) - still far from a solution.

Dataset photograph - 0718-18_Color_png.rf.7ce49fe96d1498ecbab4d952cca0c910.jpg

Classical (Milestone 1)



neighbour 0.29 position 0.06

Siamese CNN



neighbour 0.17 position 0.03

Graph neural network



neighbour 0.17 position 0.11

milestone2/figures/real_0718-18_Color_png.rf.7ce49fe96d1498ecbab.png — A third photograph. Across all eight the learned models report much better seam quality while placing no more pieces correctly, which is what being confidently wrong outside the training distribution looks like.